

Project Description

I have built a Password Strength Checker & Generator desktop application using Java and Swing. This project has two main features that work together in one program.

The first feature is a Password Checker. The user types a password into the text field, and the program instantly analyzes it. It checks things like length, uppercase letters, lowercase letters, numbers, special characters, and repeated characters. Based on all these checks, it gives a score out of 10 and shows whether the password is Weak (0-3), Medium (4-6), or Strong (7-10). The strength bar changes color from red to orange to green as the password gets stronger. A list of rules shows exactly which checks passed and which failed, so the user knows what to improve.

The second feature is a Password Generator. The user can set the desired length and choose whether to include numbers and symbols. The program then generates a random password using those settings. Once generated, the password is automatically analyzed and the strength is displayed right away. The user can also save any password to a text file for later use.

The entire project is built using object-oriented programming principles. I used inheritance with an abstract base class, polymorphism through overridden methods, encapsulation with private fields, and abstraction through the abstract class contract. The GUI is built with Java Swing components like JFrame, JPasswordField, JProgressBar, JButton, and JOptionPane. I used Java Collections like ArrayList, HashMap, and HashSet throughout the project. File handling is done with FileWriter, and I handled exceptions using try-catch-finally blocks.

Project Idea

The idea for this project came from a simple observation. Almost everyone uses passwords every day, but most people do not really know what makes a password strong or weak. I see my friends using passwords like their name or birth year, and I wanted to build something that helps people understand password security better.

I also wanted to build one complete application where all the Java concepts I learned in class come together naturally. Instead of writing small separate programs for inheritance, collections, file handling, and GUI, I wanted one project where everything fits together because it makes sense. Inheritance is not forced here - different password policies have different rules, so having a base class and subclasses is natural. Collections are not random - I genuinely need an ArrayList to store rule results, a HashMap to track password history, and a HashSet to find which character types are present. File handling is not extra - being able to

save generated passwords is a real useful feature that users would expect.

The goal was simple: build something useful, learn by doing, and show that I understand how to apply all the concepts I have studied.

Source Code

MainGUI.java

```
import javax.swing.*;
import java.awt.*;
import java.util.HashMap;

public class MainGUI {
    private JFrame frame;
    private JPasswordField passwordField;
    private JCheckBox showPasswordCheck;
    private JProgressBar strengthBar;
    private JLabel strengthLabel;
    private JPanel rulesPanel;
    private JButton checkBtn, generateBtn, saveBtn;
    private JTextField lengthField;
    private JCheckBox includeNumbersCheck, includeSymbolsCheck;

    private HashMap<String, Integer> passwordHistory;
    private PasswordGenerator generator;

    public MainGUI() {
        passwordHistory = new HashMap<>();
        generator = new PasswordGenerator();
        initUI();
    }

    private void initUI() {
        frame = new JFrame("Password Strength Checker");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout(10, 10));

        JLabel title = new JLabel("Password Strength Checker", SwingConstants.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 18));
        frame.add(title, BorderLayout.NORTH);

        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new BoxLayout(centerPanel, BoxLayout.Y_AXIS));
        centerPanel.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));

        JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        inputPanel.add(new JLabel("Enter Password:"));
        passwordField = new JPasswordField(20);
        passwordField.setEchoChar('\u2022');
        inputPanel.add(passwordField);
        showPasswordCheck = new JCheckBox("Show");
        showPasswordCheck.addActionListener(e -> {
            if (showPasswordCheck.isSelected()) {
                passwordField.setEchoChar((char) 0);
            } else {
                passwordField.setEchoChar('\u2022');
            }
        });
    }
}
```

```

inputPanel.add(showPasswordCheck);
centerPanel.add(inputPanel);

JPanel strengthPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
strengthBar = new JProgressBar(0, 10);
strengthBar.setPreferredSize(new Dimension(200, 25));
strengthBar.setStringPainted(true);
strengthPanel.add(new JLabel("Strength:"));
strengthPanel.add(strengthBar);
strengthLabel = new JLabel("Not checked");
strengthPanel.add(strengthLabel);
centerPanel.add(strengthPanel);

rulesPanel = new JPanel();
rulesPanel.setLayout(new BoxLayout(rulesPanel, BoxLayout.Y_AXIS));
rulesPanel.setBorder(BorderFactory.createTitledBorder("Rules"));
centerPanel.add(rulesPanel);

JPanel genPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
genPanel.setBorder(BorderFactory.createTitledBorder("Generator"));
genPanel.add(new JLabel("Length:"));
lengthField = new JTextField(5);
lengthField.setText("16");
genPanel.add(lengthField);
includeNumbersCheck = new JCheckBox("Numbers");
includeNumbersCheck.setSelected(true);
genPanel.add(includeNumbersCheck);
includeSymbolsCheck = new JCheckBox("Symbols");
includeSymbolsCheck.setSelected(true);
genPanel.add(includeSymbolsCheck);
centerPanel.add(genPanel);

frame.add(centerPanel, BorderLayout.CENTER);

JPanel btnPanel = new JPanel(new FlowLayout());
checkBtn = new JButton("Check");
generateBtn = new JButton("Generate");
saveBtn = new JButton("Save");
btnPanel.add(checkBtn);
btnPanel.add(generateBtn);
btnPanel.add(saveBtn);
frame.add(btnPanel, BorderLayout.SOUTH);

checkBtn.addActionListener(e -> checkPassword());
generateBtn.addActionListener(e -> generatePassword());
saveBtn.addActionListener(e -> savePassword());

frame.setSize(450, 450);
frame.setVisible(true);
frame.setLocationRelativeTo(null);
}

private void checkPassword() {
    try {
        String password = new String(passwordField.getPassword());
        if (password == null || password.isEmpty()) {
            throw new IllegalArgumentException("Password cannot be empty");
        }

        PasswordAnalyzer analyzer = new PasswordAnalyzer(password);
        int score = analyzer.getScore();
        String strength = analyzer.getStrengthLabel();
    }
}

```

```

strengthBar.setValue(score);
strengthLabel.setText(strength);

if (score <= 3) {
    strengthBar.setForeground(Color.RED);
} else if (score <= 6) {
    strengthBar.setForeground(Color.ORANGE);
} else {
    strengthBar.setForeground(Color.GREEN);
}

rulesPanel.removeAll();
for (String rule : analyzer.getRules()) {
    rulesPanel.add(new JLabel(rule));
}
rulesPanel.revalidate();
rulesPanel.repaint();

passwordHistory.put(password, score);

PasswordPolicy weak = new WeakPolicy();
PasswordPolicy strong = new StrongPolicy();
boolean weakValid = weak.validate(password);
boolean strongValid = strong.validate(password);

String msg = "Score: " + score + "/10 (" + strength + ")\n";
msg += "HashSet types found: " + analyzer.getTypesFound() + "\n";
msg += "History entries: " + passwordHistory.size() + "\n";
msg += "WeakPolicy: " + (weakValid ? "Valid" : "Invalid") + "\n";
msg += "StrongPolicy: " + (strongValid ? "Valid" : "Invalid");

JOptionPane.showMessageDialog(frame, msg, "Analysis Result",
JOptionPane.INFORMATION_MESSAGE);

    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(frame, e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    } finally {
        strengthBar.repaint();
    }
}

private void generatePassword() {
    try {
        int length = Integer.parseInt(lengthField.getText().trim());
        if (length <= 0) throw new NumberFormatException();

        generator.includeNumbers(includeNumbersCheck.isSelected());
        generator.includeSymbols(includeSymbolsCheck.isSelected());

        String generated = generator.generate(length);
        passwordField.setText(generated);
        passwordField.setEchoChar((char) 0);
        showPasswordCheck.setSelected(true);

        PasswordAnalyzer analyzer = new PasswordAnalyzer(generated);
        int score = analyzer.getScore();
        String strength = analyzer.getStrengthLabel();

        strengthBar.setValue(score);
        strengthLabel.setText(strength);
    }
}

```

```

        if (score <= 3) {
            strengthBar.setForeground(Color.RED);
        } else if (score <= 6) {
            strengthBar.setForeground(Color.ORANGE);
        } else {
            strengthBar.setForeground(Color.GREEN);
        }

        rulesPanel.removeAll();
        for (String rule : analyzer.getRules()) {
            rulesPanel.add(new JLabel(rule));
        }
        rulesPanel.revalidate();
        rulesPanel.repaint();

        JOptionPane.showMessageDialog(frame,
            "Generated: " + generated + "\nStrength: " + strength + " (" + score +
"/10)",
            "Generated Password", JOptionPane.INFORMATION_MESSAGE);

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(frame, "Invalid length. Enter a positive
number.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

private void savePassword() {
    try {
        String password = new String(passwordField.getPassword());
        if (password == null || password.isEmpty()) {
            throw new IllegalArgumentException("Nothing to save");
        }
        generator.saveToFile(password);
        JOptionPane.showMessageDialog(frame, "Password saved to
saved_passwords.txt", "Saved", JOptionPane.INFORMATION_MESSAGE);
    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(frame, e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    } catch (Exception e) {
        JOptionPane.showMessageDialog(frame, "Save failed: " + e.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(MainGUI::new);
}
}

```

PasswordAnalyzer.java

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;

public class PasswordAnalyzer {
    private String _password;
    private int _score;
    private ArrayList<String> _rules;

```

```

private HashMap<String, Integer> _history;
private HashSet<String> _typesFound;

public PasswordAnalyzer(String password) {
    this._password = password;
    this._rules = new ArrayList<>();
    this._history = new HashMap<>();
    this._typesFound = new HashSet<>();
    analyze();
}

private void analyze() {
    _rules.clear();
    _score = 0;

    if (_password == null || _password.isEmpty()) {
        _rules.add("Password is empty");
        return;
    }

    _typesFound.clear();
    for (char c : _password.toCharArray()) {
        if (Character.isUpperCase(c)) _typesFound.add("UPPER");
        else if (Character.isLowerCase(c)) _typesFound.add("LOWER");
        else if (Character.isDigit(c)) _typesFound.add("DIGIT");
        else _typesFound.add("SPECIAL");
    }

    if (_password.length() >= 8) { _rules.add("Min 8 characters"); _score++; }
    else { _rules.add("Min 8 characters"); }

    if (_password.length() >= 12) { _rules.add("Min 12 characters"); _score++; }
    else { _rules.add("Min 12 characters"); }

    if (_password.length() >= 16) { _rules.add("Min 16 characters"); _score++; }
    else { _rules.add("Min 16 characters"); }

    if (_password.matches(".*[a-z].*")) { _rules.add("Has lowercase letter");
    _score++; }
    else { _rules.add("Has lowercase letter"); }

    if (_password.matches(".*[A-Z].*")) { _rules.add("Has uppercase letter");
    _score++; }
    else { _rules.add("Has uppercase letter"); }

    if (_password.matches(".*[0-9].*")) { _rules.add("Has number"); _score++; }
    else { _rules.add("Has number"); }

    if (_password.matches(".*[!@#$%^&*()_+\\-=\\[\\]\\{};':\"\\\\\\\\|,.<>\\\\/?.*") {
        _rules.add("Has special character"); _score++;
    } else {
        _rules.add("Has special character");
    }

    if (_typesFound.size() >= 3) { _rules.add("3+ character types"); _score++; }
    else { _rules.add("3+ character types"); }

    if (_typesFound.size() >= 4) { _rules.add("All 4 character types"); _score++; }
    else { _rules.add("All 4 character types"); }

    boolean noRepeat = true;
    for (int i = 1; i < _password.length(); i++) {

```

```

        if (_password.charAt(i) == _password.charAt(i - 1)) {
            noRepeat = false;
            break;
        }
    }
    if (noRepeat) { _rules.add("No repeated consecutive chars"); _score++; }
    else { _rules.add("No repeated consecutive chars"); }

    _history.put(_password, _score);
}

public int getScore() { return _score; }

public ArrayList<String> getRules() { return _rules; }

public HashMap<String, Integer> getHistory() { return _history; }

public HashSet<String> getTypesFound() { return _typesFound; }

public String getStrengthLabel() {
    if (_score <= 3) return "Weak";
    if (_score <= 6) return "Medium";
    return "Strong";
}
}

```

PasswordGenerator.java

```

import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

public class PasswordGenerator {
    private static final String LETTERS =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    private static final String NUMBERS = "0123456789";
    private static final String SYMBOLS = "!@#$%^&*()_+-=[]{}|;:,.<>?";

    private boolean includeNumbers;
    private boolean includeSymbols;
    private Random random;

    public PasswordGenerator() {
        this.includeNumbers = true;
        this.includeSymbols = true;
        this.random = new Random();
    }

    public void includeNumbers(boolean include) { this.includeNumbers = include; }
    public void includeSymbols(boolean include) { this.includeSymbols = include; }

    public String generate(int length) {
        StringBuilder pool = new StringBuilder(LETTERS);
        if (includeNumbers) pool.append(NUMBERS);
        if (includeSymbols) pool.append(SYMBOLS);

        StringBuilder password = new StringBuilder();
        for (int i = 0; i < length; i++) {
            int index = random.nextInt(pool.length());
            password.append(pool.charAt(index));
        }
    }
}

```

```

    }
    return password.toString();
}

public void saveToFile(String password) throws IOException {
    FileWriter fw = null;
    try {
        fw = new FileWriter("saved_passwords.txt", true);
        fw.write(password + System.lineSeparator());
    } finally {
        if (fw != null) {
            try { fw.close(); } catch (IOException e) {
                System.err.println("Error closing file: " + e.getMessage());
            }
        }
    }
}
}

```

PasswordPolicy.java

```

public abstract class PasswordPolicy {
    protected int minLength;
    protected boolean requireNumbers;
    protected boolean requireSpecialChars;

    public abstract boolean validate(String password);

    public int getMinLength() { return minLength; }
    public boolean isRequireNumbers() { return requireNumbers; }
    public boolean isRequireSpecialChars() { return requireSpecialChars; }
}

```

StrongPolicy.java

```

public class StrongPolicy extends PasswordPolicy {
    public StrongPolicy() {
        this.minLength = 12;
        this.requireNumbers = true;
        this.requireSpecialChars = true;
    }

    @Override
    public boolean validate(String password) {
        if (password == null || password.length() < minLength) return false;
        if (requireNumbers && !password.matches(".*[0-9].*")) return false;
        if (requireSpecialChars &&
!password.matches(".*[!@#$%^&*()_+\\-=\\[\\]\\{};':\"\\\\\\\\|,.<>\\\\/?.*")) return false;
        if (!password.matches(".*[A-Z].*")) return false;
        return true;
    }
}

```

WeakPolicy.java

```

public class WeakPolicy extends PasswordPolicy {
    public WeakPolicy() {
        this.minLength = 6;
    }
}

```



```

        this.requireNumbers = false;
        this.requireSpecialChars = false;
    }

    @Override
    public boolean validate(String password) {
        if (password == null) return false;
        return password.length() >= minLength;
    }
}

```

What I Learned

By building this project, I learned several important concepts that I had only read about in textbooks before:

- Abstract classes are not just a theoretical concept. I used `PasswordPolicy` as an abstract class and it forced me to implement `validate()` in both `WeakPolicy` and `StrongPolicy`. If I forget to override it, the compiler gives an error. This made me understand why abstraction is useful in real projects.
- Polymorphism works exactly as described in class. When I write `PasswordPolicy p = new StrongPolicy()` and call `p.validate()`, Java automatically calls `StrongPolicy validate()` version. This is runtime polymorphism and I actually saw it working in my program.
- Encapsulation is about protecting data, not just hiding it. My private fields with public getters ensure that no other class can directly change the password or score. The getters act as a controlled interface to access the data.
- Swing is more complex than I expected. Managing layouts with `BorderLayout`, `BoxLayout`, and `FlowLayout` together took some experimentation. I also learned that `JPasswordField` has `setEchoChar()` for hiding characters, which `JTextField` does not have.
- File handling with `FileWriter` is straightforward but needs careful resource management. The finally block is important for closing files. I learned that wrapping `fw.close()` in its own try-catch is necessary because `close()` itself can throw an exception.
- Choosing the right Collection makes the code cleaner. `ArrayList` works perfectly for ordered rule results. `HashMap` is ideal for storing password-score pairs. `HashSet` automatically handles uniqueness when tracking character types.
- Breaking a problem into multiple classes with single responsibilities makes the code easier to write, test, and debug. Each of my 6 classes has one clear job and they connect through well-defined methods and constructors.
- Exception handling prevents the program from crashing. Instead of getting a console error, the user sees a friendly dialog box explaining what went wrong. This makes the application feel professional and complete.

Conclusion

I successfully built a Password Strength Checker & Generator desktop application using Java and Swing. The project works as intended - users can check the strength of any password and generate strong random passwords with customizable options.

I enjoyed building this project because it gave me a chance to apply everything I learned in class to a real, working application. Seeing the code I wrote turn into a functional GUI window that analyzes passwords and generates secure ones was very satisfying. The project also taught me how to plan the structure of an application before writing code, which is something I will definitely do in future projects.

If I had more time, I would add more features like a password strength history chart, export to CSV, support for dark mode, and maybe a simple database instead of a text file. But for now, this project meets all my requirements and I am happy with the result.

Outputs

`saved_passwords.txt`

```
] :.+h%z+i}O5Y(Eq
```



